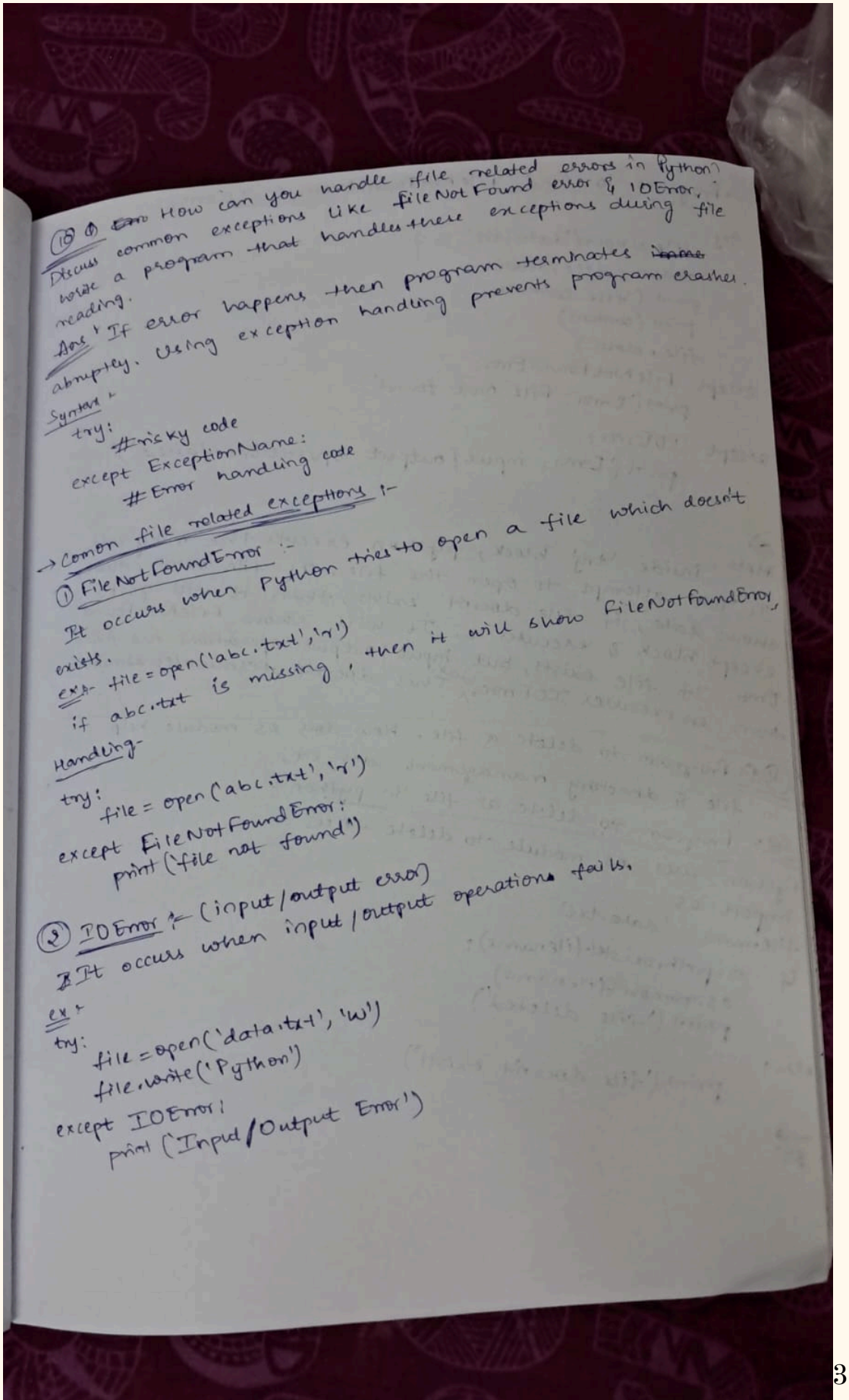


(10) How can you handle file related errors in Python? Discuss common exceptions like FileNotFound error & IOError, while a program that handles these exceptions during file reading.

Ans: If error happens then program terminates abruptly. Using exception handling prevents program crashes.

Syntax:-

```
try:
    #risky code
except ExceptionName:
    #Error handling code
```

→ Comon file related exceptions :-

① FileNotFoundError :-

It occurs when Python tries to open a file which doesn't exists.

ex:- file = open('abc.txt','r')

if abc.txt is missing, then it will show FileNotFoundError,

Handling-

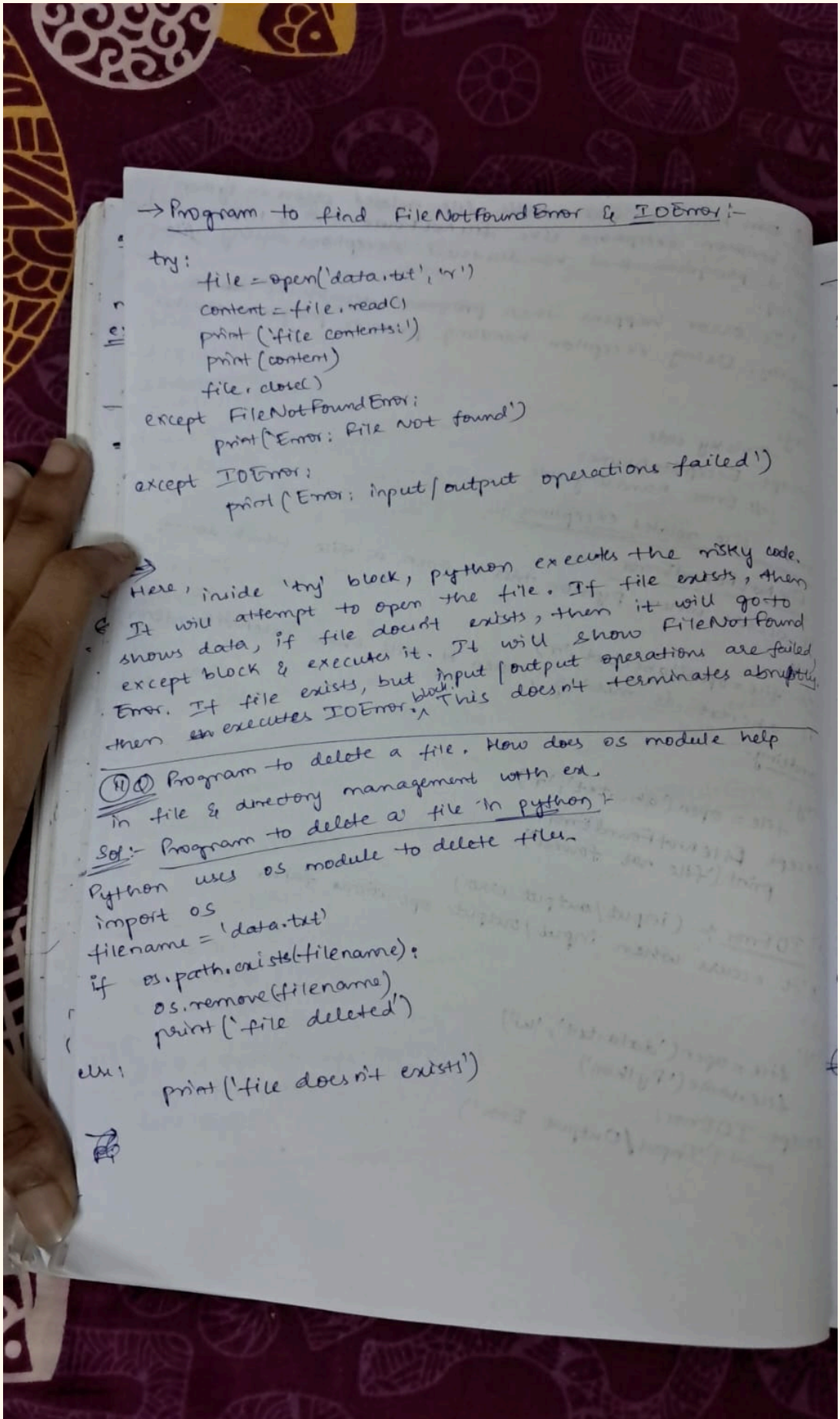
```
try:
    file = open('abc.txt','r')
except FileNotFoundError:
    print('file not found')
```

② IOError :- (input/output error)

It occurs when input/output operations fails.

ex:-

```
try:
    file = open('data.txt', 'w')
    file.write('Python')
except IOError:
    print('Input/Output Error')
```



→ Program to find FileNotFoundError & IOError :-

```
try:
    file = open('data.txt', 'r')
    content = file.read()
    print ('file contents:')
    print (content)
    file.close()
except FileNotFoundError:
    print ('Error: File not found')
except IOError:
    print ('Error: input/output operations failed')
```

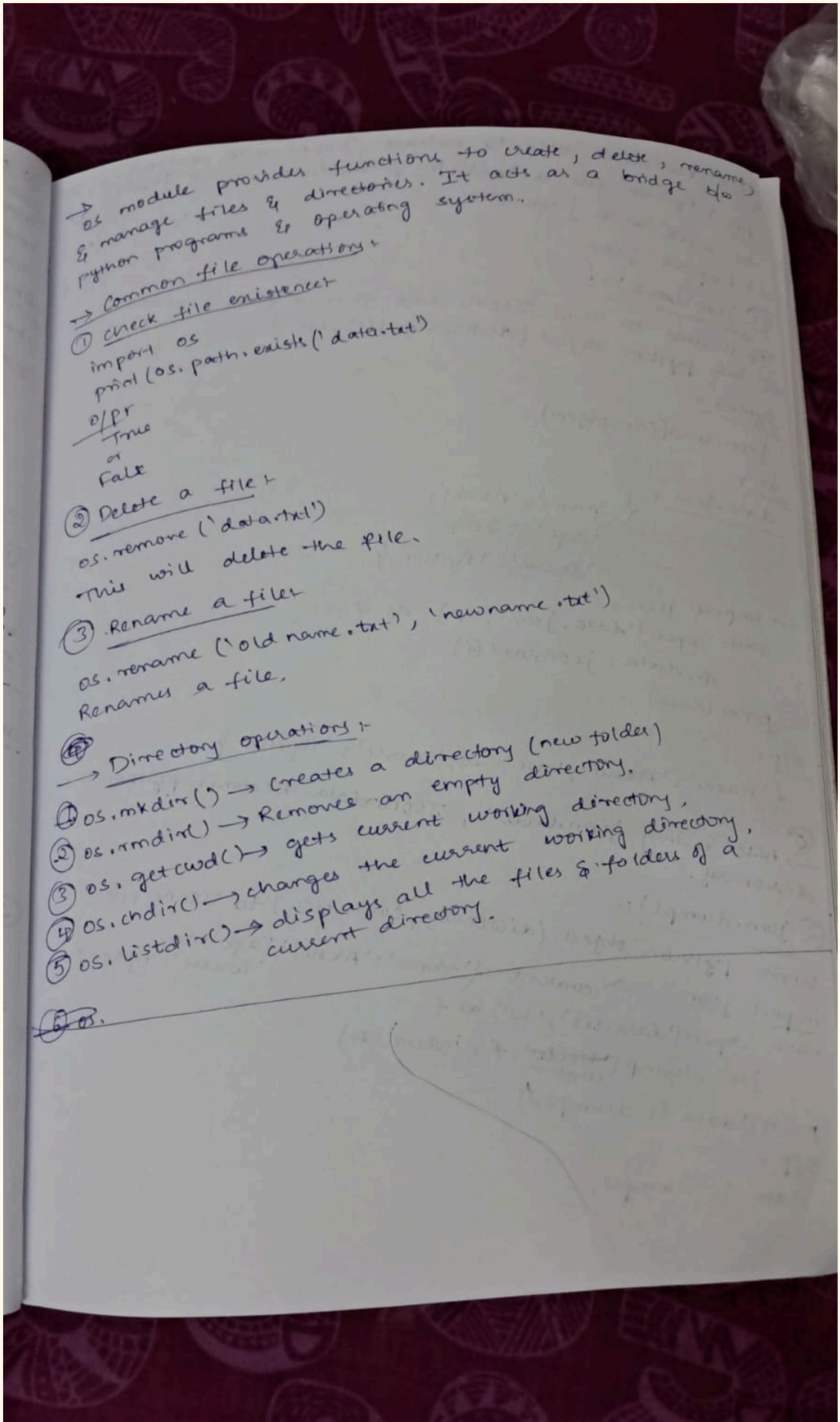
Here, inside 'try' block, python executes the risky code. It will attempt to open the file. If file exists, then shows data, if file doesn't exists, then it will go to except block & executes it. It will show FileNotFound Error. If file exists, but input/output operations are failed then executes IOError. This doesn't terminates abruptly.

(11)(a) Program to delete a file. How does os module help in file & directory management with ex.

Sol:- Program to delete a file in python :-

Python uses os module to delete files.

```
import os
filename = 'data.txt'
if os.path.exists(filename):
    os.remove(filename)
    print ('file deleted')
else:
    print ('file doesn't exists')
```



→ os module provides functions to create, delete, rename & manage files & directories. It acts as a bridge b/w python programs & operating system.

→ Common file operations:

① Check file existence:

```
import os
print(os.path.exists('data.txt'))
```

o/p
True
or
False

② Delete a file:

```
os.remove('data.txt')
```

This will delete the file.

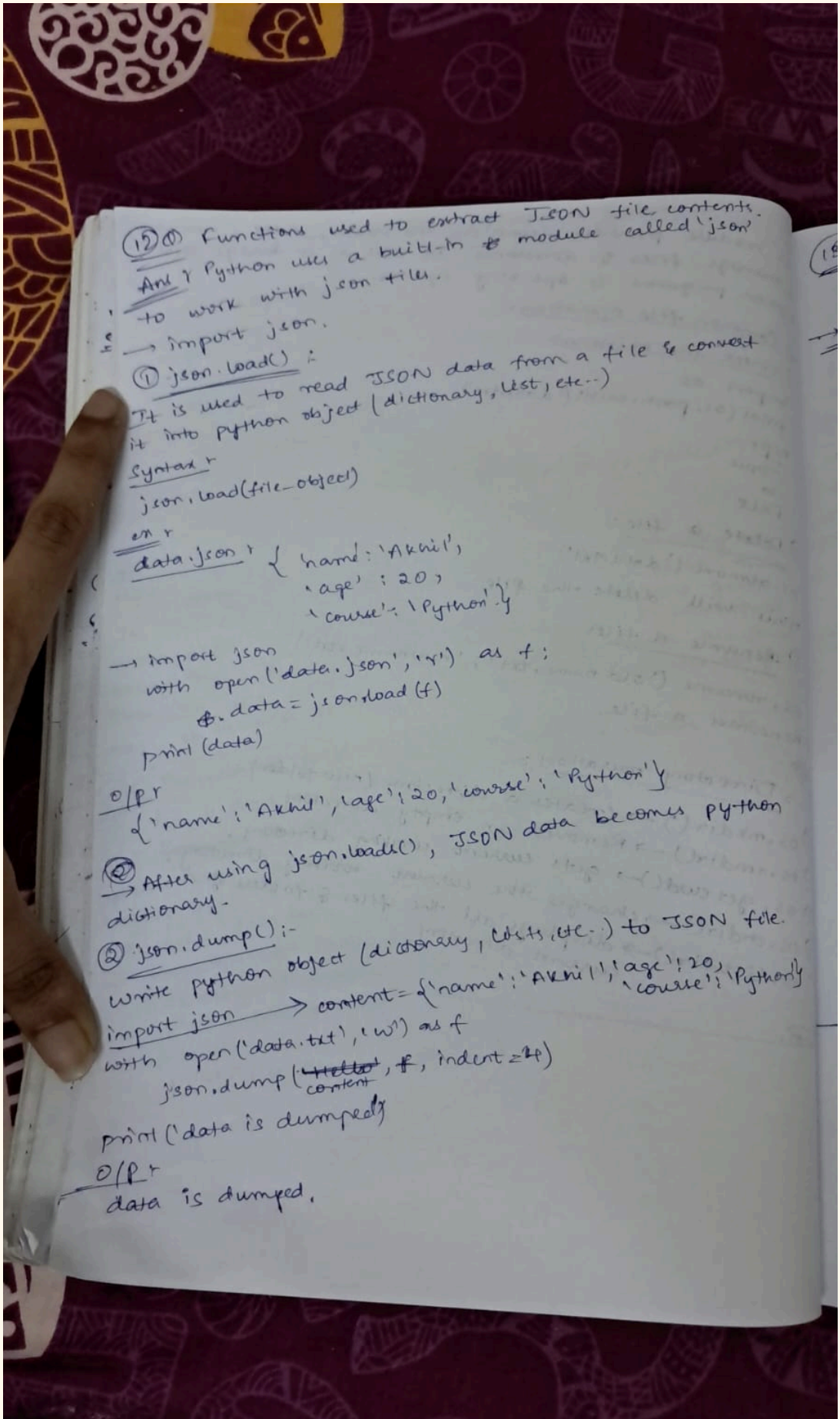
③ Rename a file:

```
os.rename('old name.txt', 'new name.txt')
```

Renames a file.

→ Directory operations:

① os.mkdir() → creates a directory (new folder)
② os.rmdir() → Removes an empty directory.
③ os.getcwd() → gets current working directory.
④ os.chdir() → changes the current working directory.
⑤ os.listdir() → displays all the files & folders of a current directory.



(12) (1) Functions used to extract JSON file contents.

Ans: Python uses a built-in module called 'json' to work with json files.

→ import json.

① json.load() :-

It is used to read JSON data from a file & convert it into python object (dictionary, list, etc--)

Syntax:

```
json.load(file_object)
```

ex:

data.json: { 'name': 'Akhil',
'age': 20,
'course': 'Python' }

```
import json
with open('data.json', 'r') as f:
    data = json.load(f)
print(data)
```

O/P:

{'name': 'Akhil', 'age': 20, 'course': 'Python'}

→ After using json.loads(), JSON data becomes python dictionary.

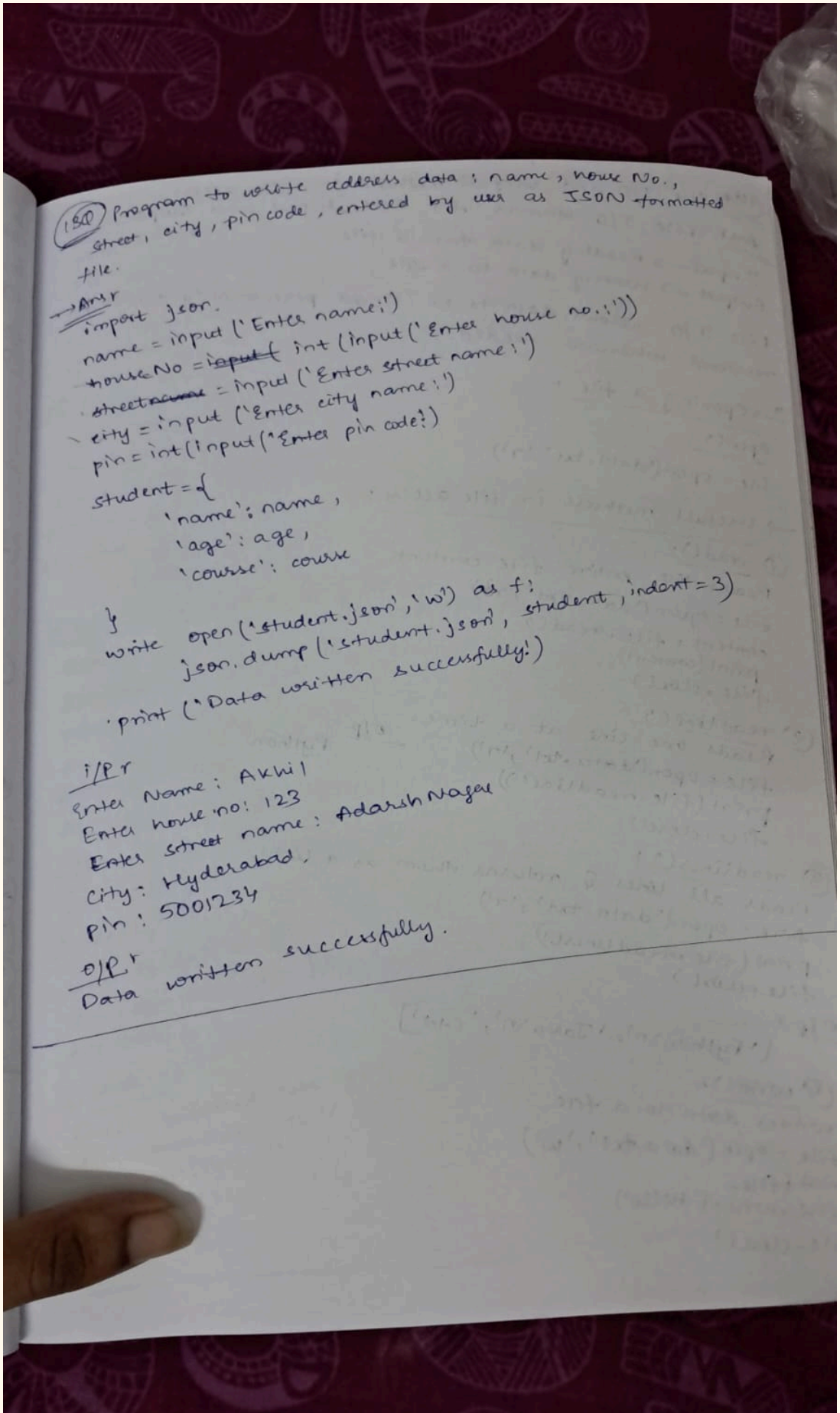
② json.dump() :-

write python object (dictionary, lists, etc--) to JSON file.

```
import json
content = {'name': 'Akhil', 'age': 20, 'course': 'Python'}
with open('data.txt', 'w') as f
    json.dump(content, f, indent=4)
print('data is dumped')
```

O/P:

data is dumped.



(15) Program to write address data : name, house No., street, city, pin code, entered by user as JSON formatted file.

→ Ans r

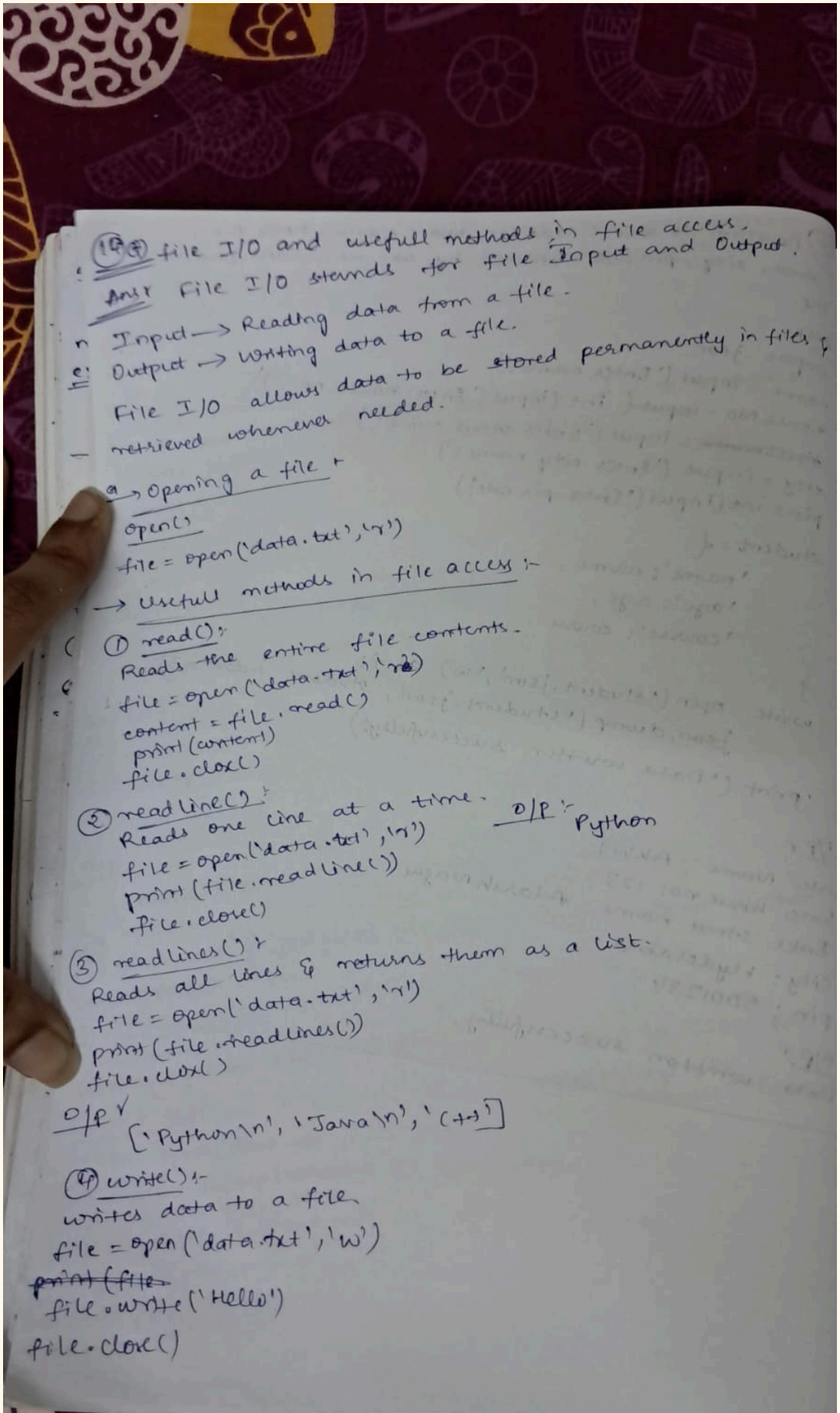
```
import json.
name = input ('Enter name:')
houseNo = int (input ('Enter house no.:'))
street = input ('Enter street name:')
city = input ('Enter city name:')
pin = int(input('Enter pin code:')
student = {
    'name': name,
    'age': age,
    'course': course
}
write open ('student.json', 'w') as f:
    json.dump ('student.json', student, indent=3)
print ('Data written successfully!')
```

i/p r

Enter Name : Akhil
Enter house no: 123
Enter street name : Adarsh Nagar
City : Hyderabad.
Pin : 5001234

o/p r

Data written successfully.



(19) file I/O and usefull methods in file access.

Ans: File I/O stands for file Input and Output.

Input → Reading data from a file.

Output → Writing data to a file.

File I/O allows data to be stored permanently in files & retrieved whenever needed.

→ Opening a file :-

open()

file = open('data.txt','r')

→ Usefull methods in file access :-

① read():

Reads the entire file contents.

```
file = open('data.txt','r')
content = file.read()
print(content)
file.close()
```

② readline():

Reads one line at a time.

```
file = open('data.txt','r')
print(file.readline())
file.close()
```

O/P :- Python

③ readlines():

Reads all lines & returns them as a list.

```
file = open('data.txt','r')
print(file.readlines())
file.close()
```

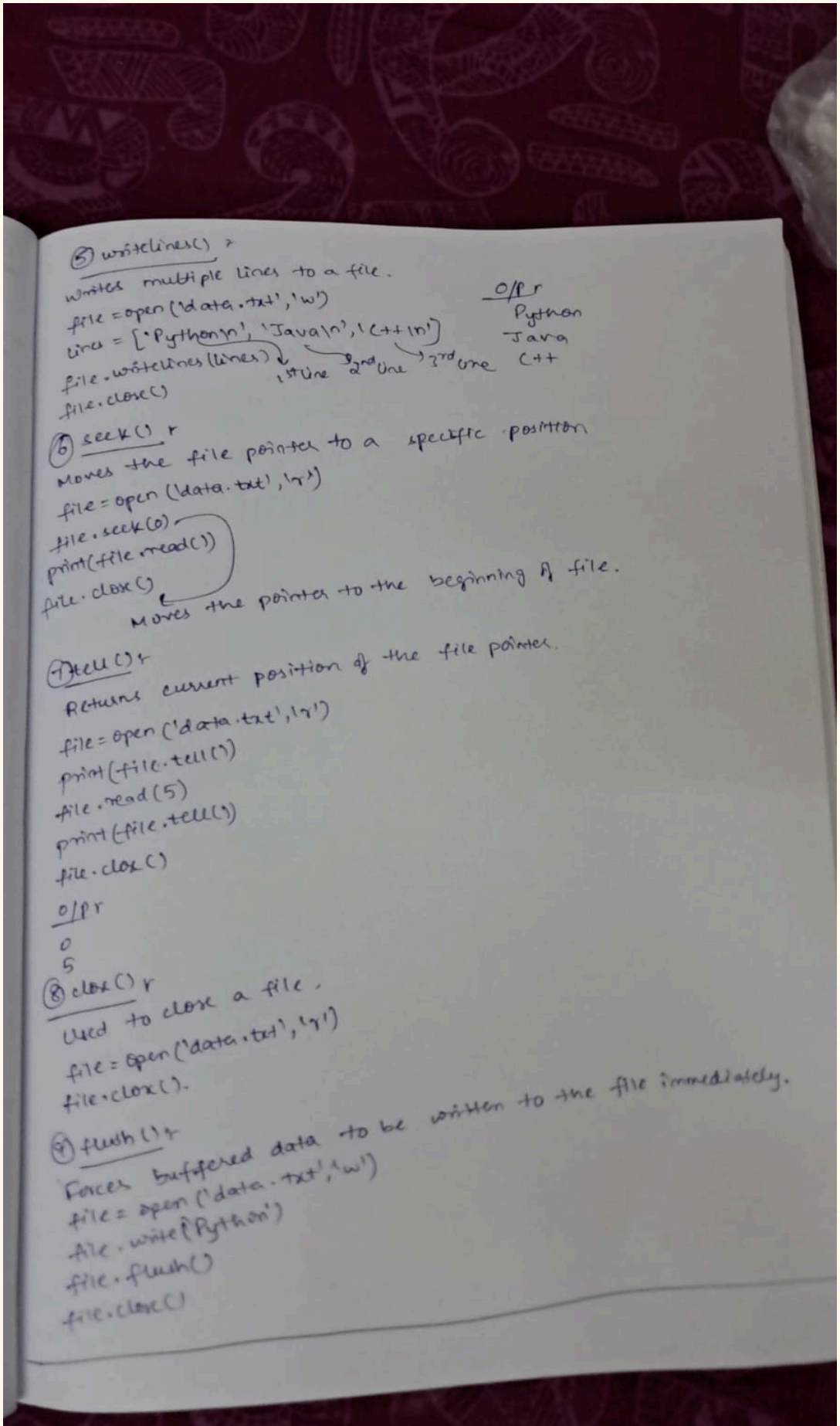
O/P :-

['Python\n', 'Java\n', 'C++']

④ write():-

writes data to a file

```
file = open('data.txt','w')
file.write('Hello')
file.close()
```



5) writelines() :
Writes multiple lines to a file.

```
file = open('data.txt','w')
lines = ['Python\n', 'Java\n', 'C++\n']
file.writelines(lines)
file.close()
```

1st line, 2nd line, 3rd line

O/P :
Python
Java
C++

6) seek() :
Moves the file pointer to a specific position

```
file = open('data.txt','r')
file.seek(0)
print(file.read())
file.close()
```

Moves the pointer to the beginning of file.

7) tell() :
Returns current position of the file pointer.

```
file = open('data.txt','r')
print(file.tell())
file.read(5)
print(file.tell())
file.close()
```

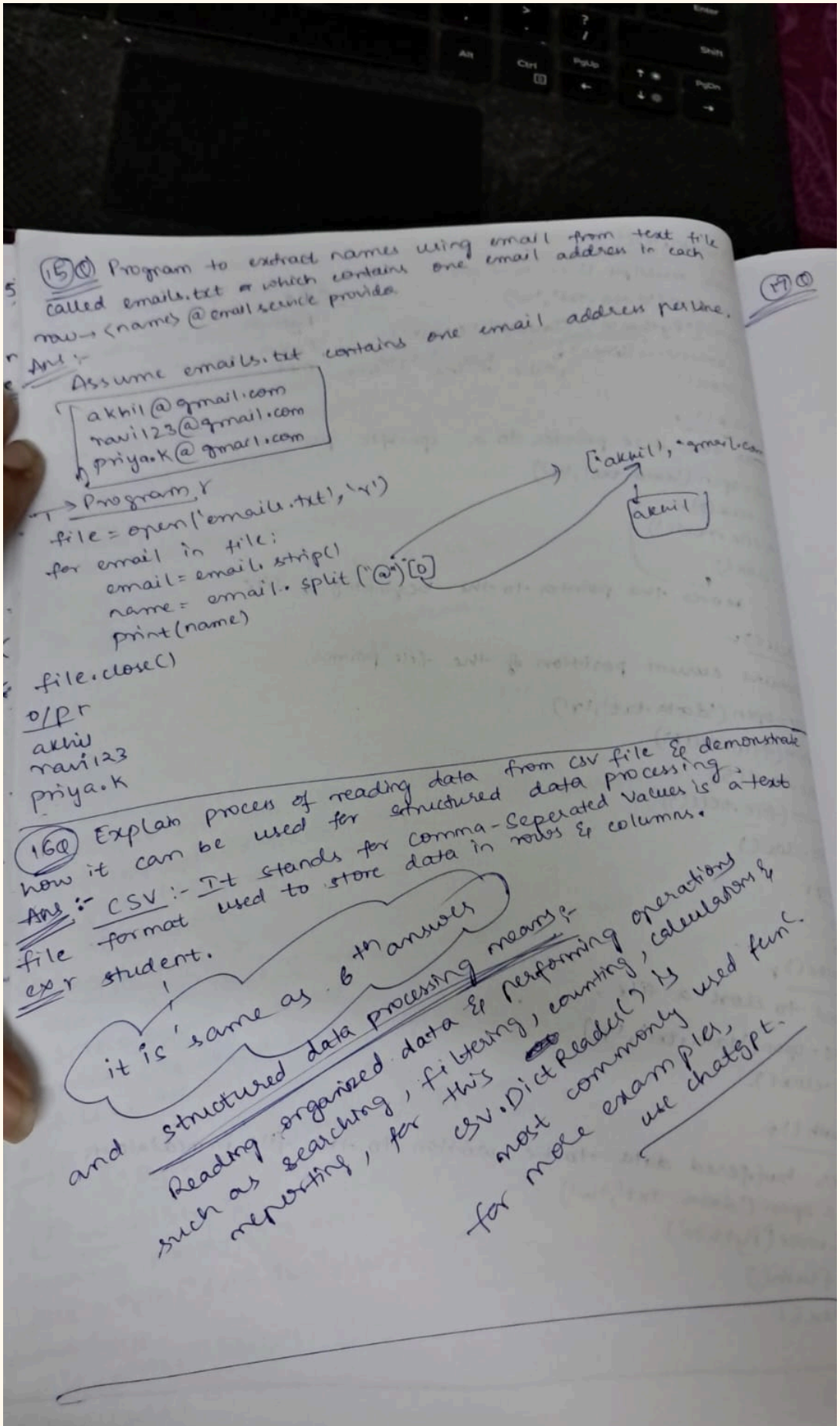
O/P :
0
5

8) close() :
Used to close a file.

```
file = open('data.txt','r')
file.close().
```

9) flush() :
Forces buffered data to be written to the file immediately.

```
file = open('data.txt','w')
file.write('Python')
file.flush()
file.close()
```



15) Program to extract names using email from text file called emails.txt which contains one email address in each row → (name)@email service provide.

Ans :-

Assume emails.txt contains one email address per line.

akhil@gmail.com
ravi123@gmail.com
priya.k@gmail.com

Program :-

```
file = open('emails.txt', 'r')
for email in file:
    email = email.strip()
    name = email.split("@")[0]
    print(name)
file.close()
```

['akhil', 'gmail.com'] → akhil

o/p :-

akhil
ravi123
priya.k

16) Explain process of reading data from csv file & demonstrate how it can be used for structured data processing.

Ans :- CSV :- It stands for comma-seperated values is a text file format used to store data in rows & columns.

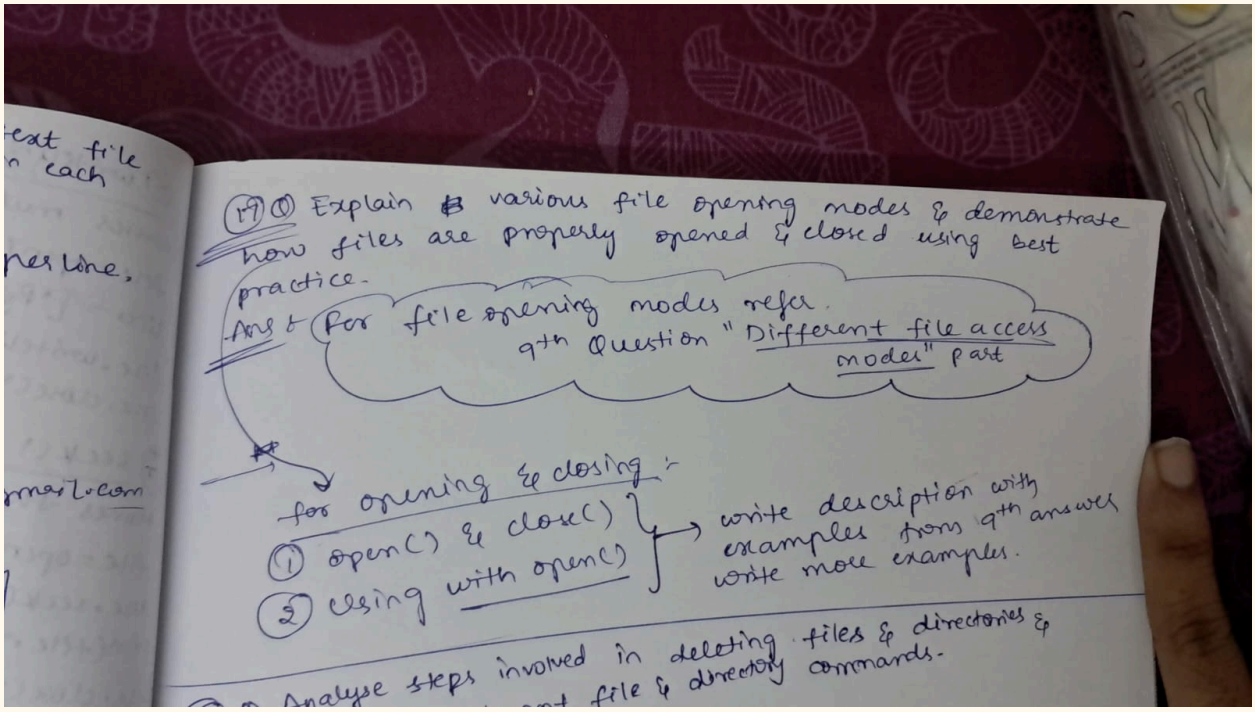
ex :- student.

it is same as 6th answer

and structured data processing means :-

Reading organized data & performing operations such as searching, filtering, counting, calculations & reporting, for this csv.DictReader() is most commonly used func.

for more examples, use chatgpt.



(19) (1) Explain various file opening modes & demonstrate how files are properly opened & closed using best practice.

Ans:- For file opening modes refer 9th Question "Different file access modes" part

for opening & closing :-

1. open() & close()
2. using with open()

→ write description with examples from 9th answer write more examples.

(2) Analyse steps involved in deleting files & directories & file & directory commands.